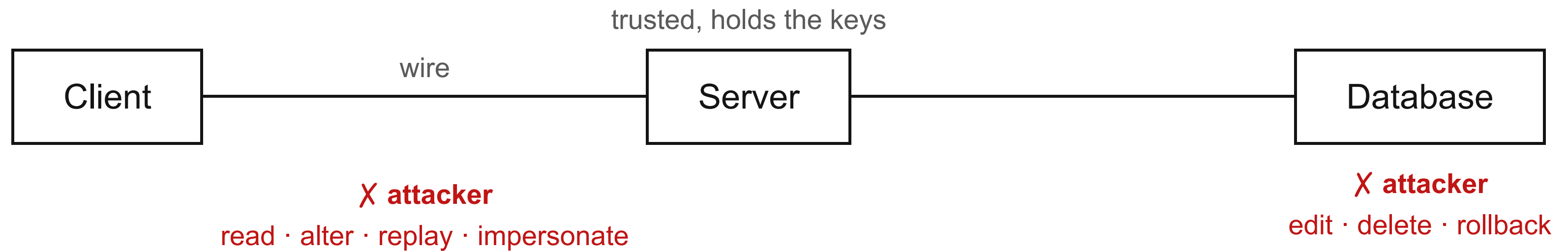


Implementing security for a social network

DUCAFilms · Foundations of Cybersecurity 2025–2026

Luca Di Gasparro · Mykhaylo Semenyshen

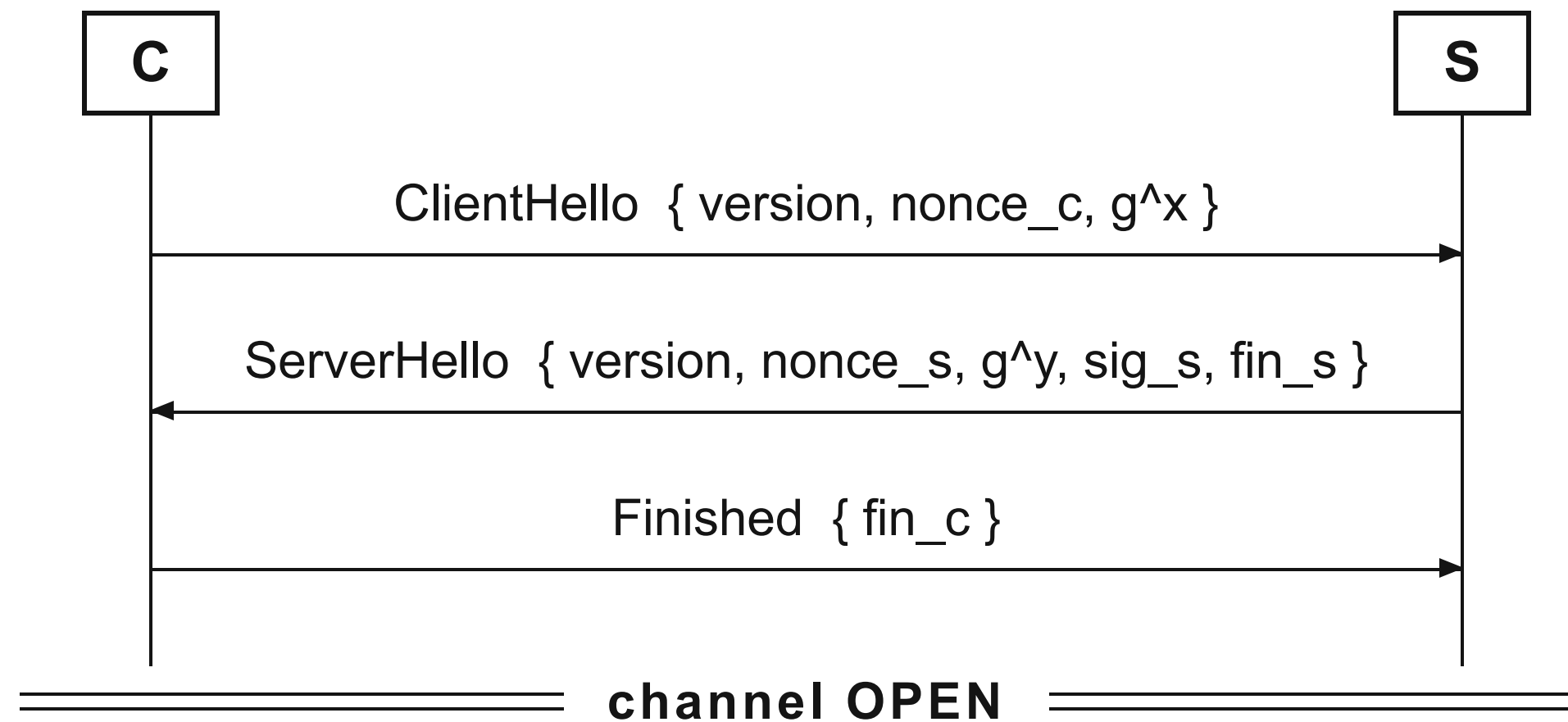
An attacker on the wire, an attacker in the database



Four mechanisms cover the two attackers

Secure channel	confidentiality · integrity · non-malleability · replay-resistance · forward secrecy · server auth
Session token	user authentication · authorization · revocation
Signed reviews	authenticity at rest · offline verifiability
Merkle log	deletion & reordering detection

One round trip establishes keys and authenticates the server



transcript = version || nonce_c || g^x || nonce_s || g^y

sig_s = Sig_privKc(transcript) (server authentication)

fin_s = HMAC_khs(transcript) (key confirmation)

fin_c = HMAC_khc(transcript || sig_s || fin_s) (key confirmation)

key exchange → X25519 (ECDH)

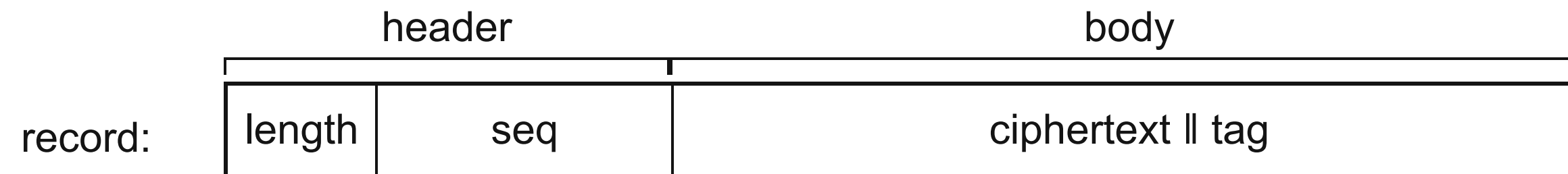
key derivation → HKDF-SHA256

MAC → HMAC-SHA256

hash → SHA-256

signature → Ed25519

Sequence numbers inside the AEAD stop replay and reordering



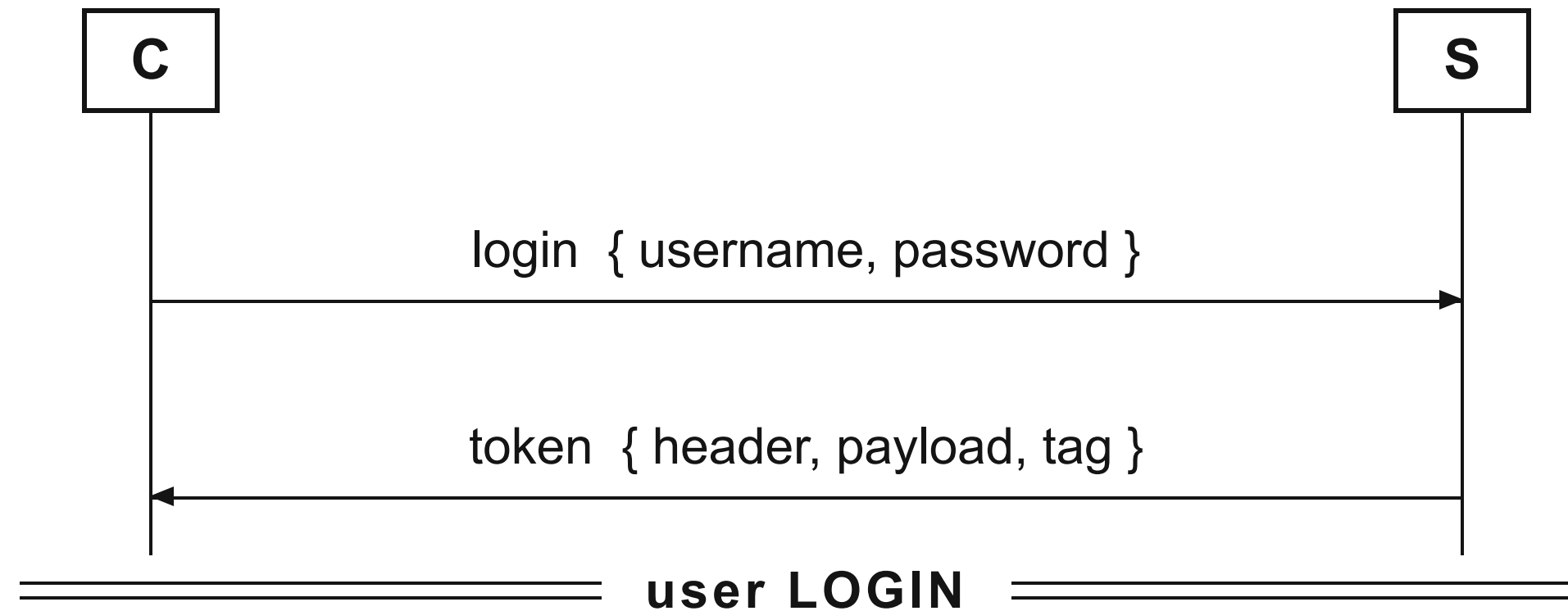
ciphertext, tag = AEAD(key, nonce, plaintext, AAD)

nonce = iv $\oplus$ seq

AAD = version || length || seq

AEAD $\rightarrow$ AES-256-GCM

Login issues a session token bound to the channel



header = {algorithm, type}

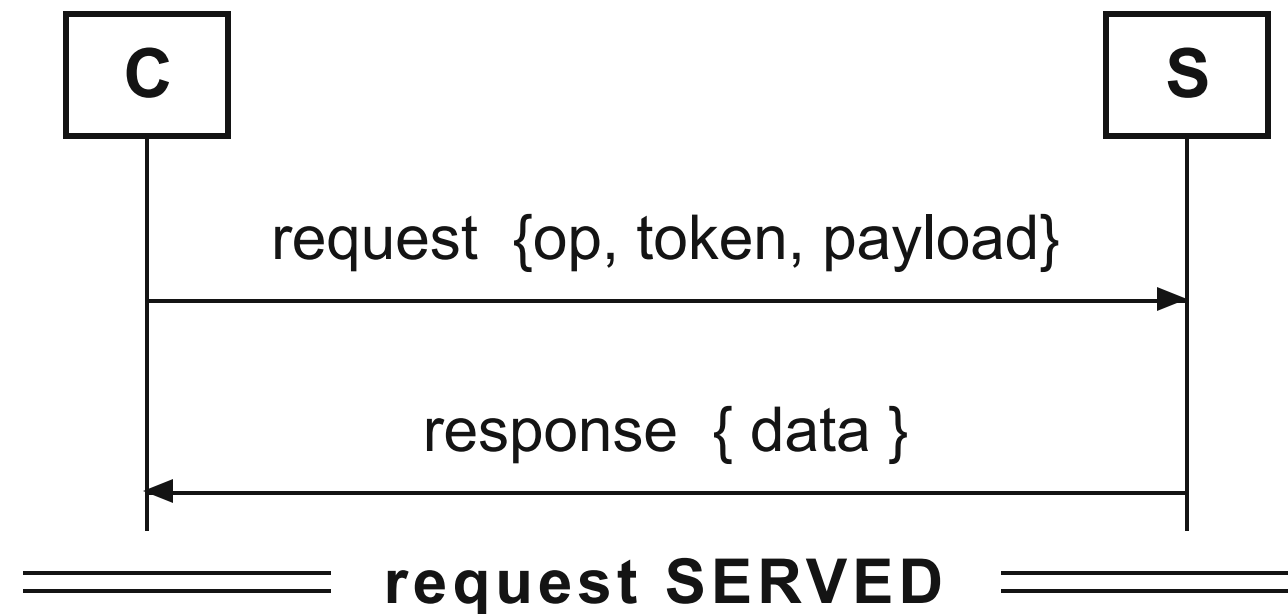
payload = {user_id, token_id, channel_id, privileges,
validity window, issuer, audience}

tag = MAC_K(header, payload)

MAC → HMAC-SHA256

K → server's private key for tokens

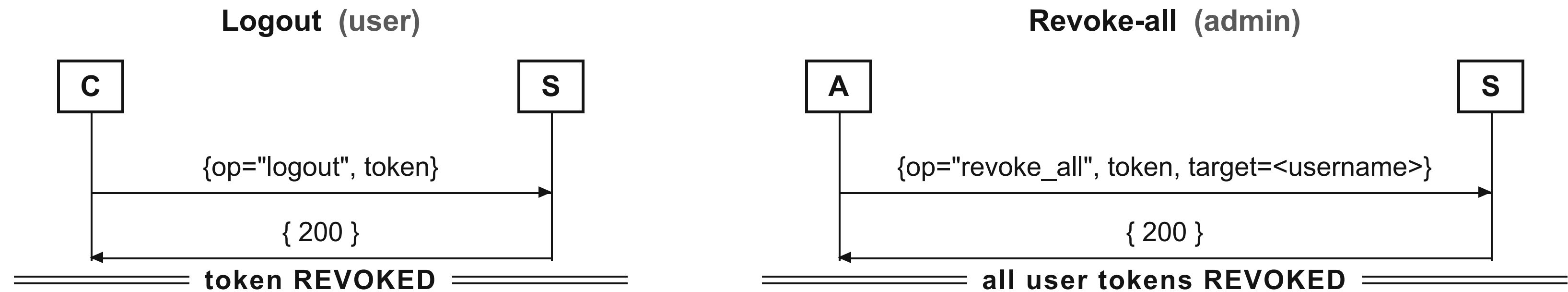
Each request re-validates its token before being served



Validation Pipeline

- ❑ Reject algorithm $\neq$ HS256
- ❑ Verify HMAC tag
- ❑ Check issuer & audience
- ❑ Check validity window
- ❑ Check revocation
- ❑ Verify channel
- ❑ Check privileges

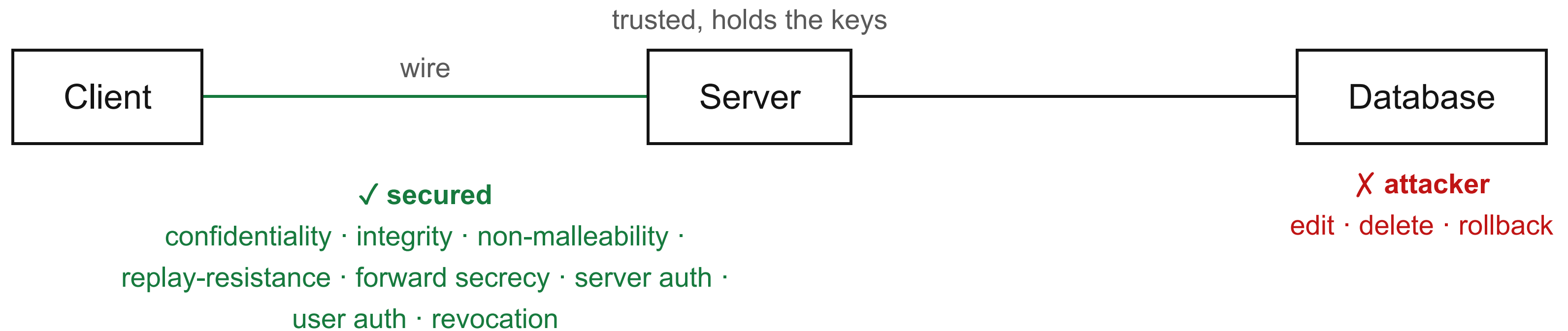
Tokens are stateless except for revocation set



REVSET ENTRY

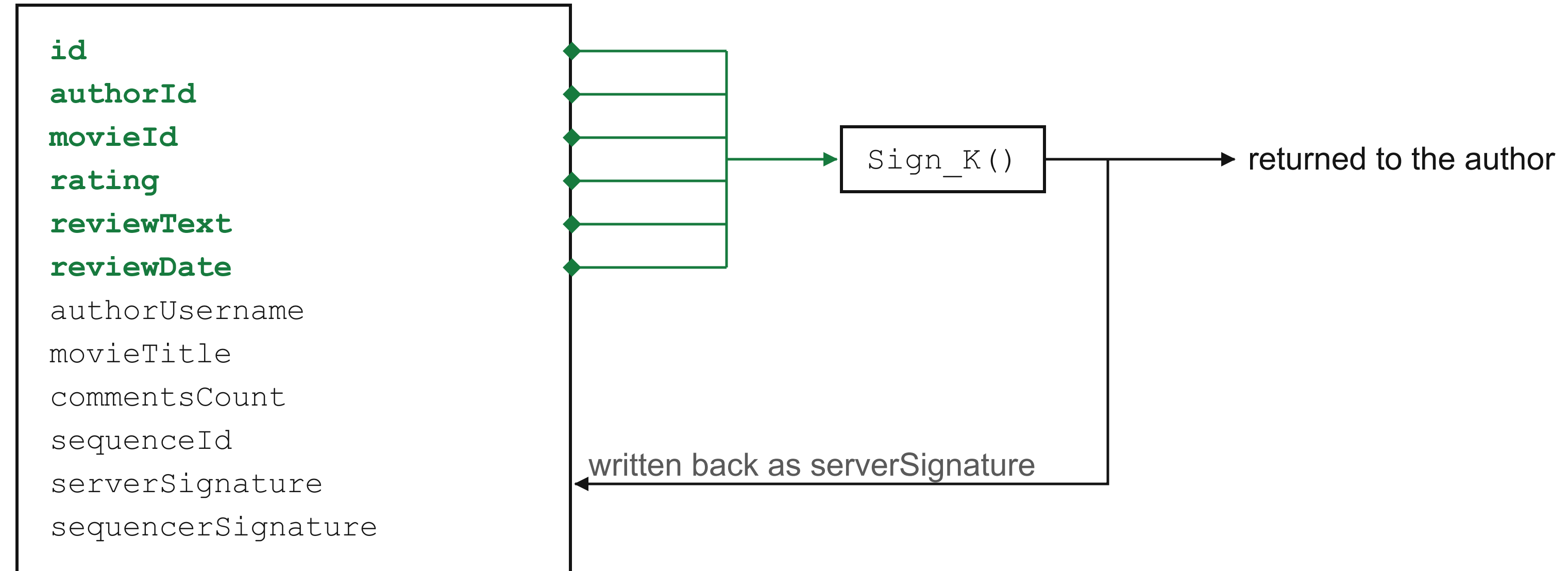
scope	TOKEN USER
key	token_id user_id
issued_at	when revoked
expire_at	issued_at + TTL

The wire is secured; the database is still exposed



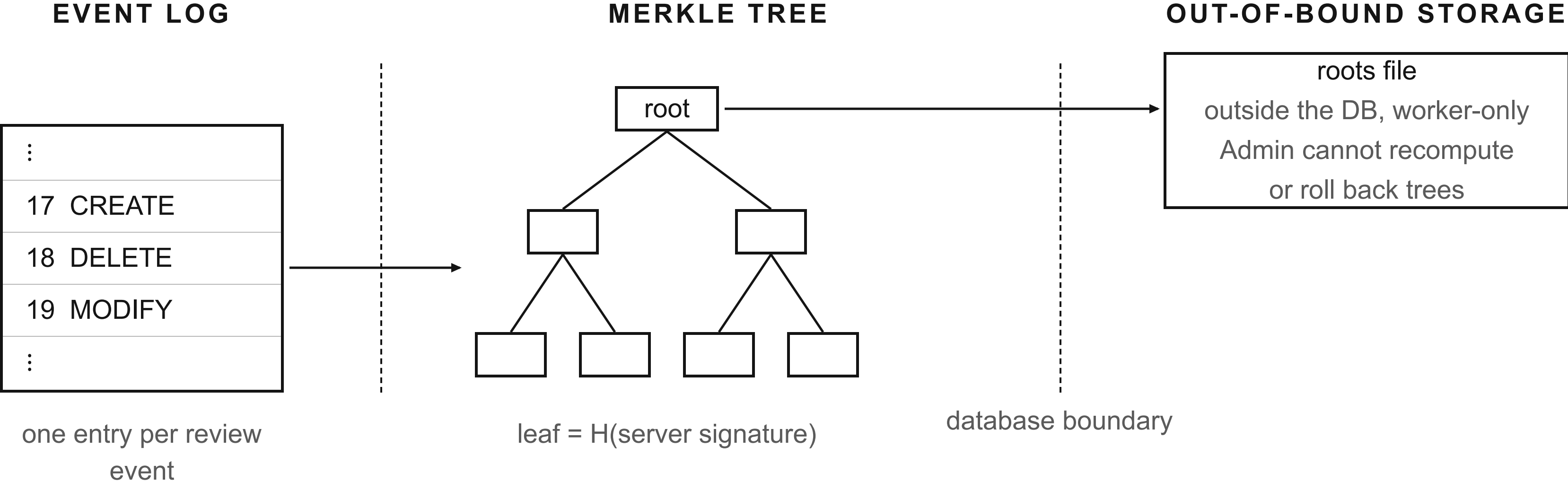
Editing any signed field breaks the review's signature

REVIEW DOCUMENT

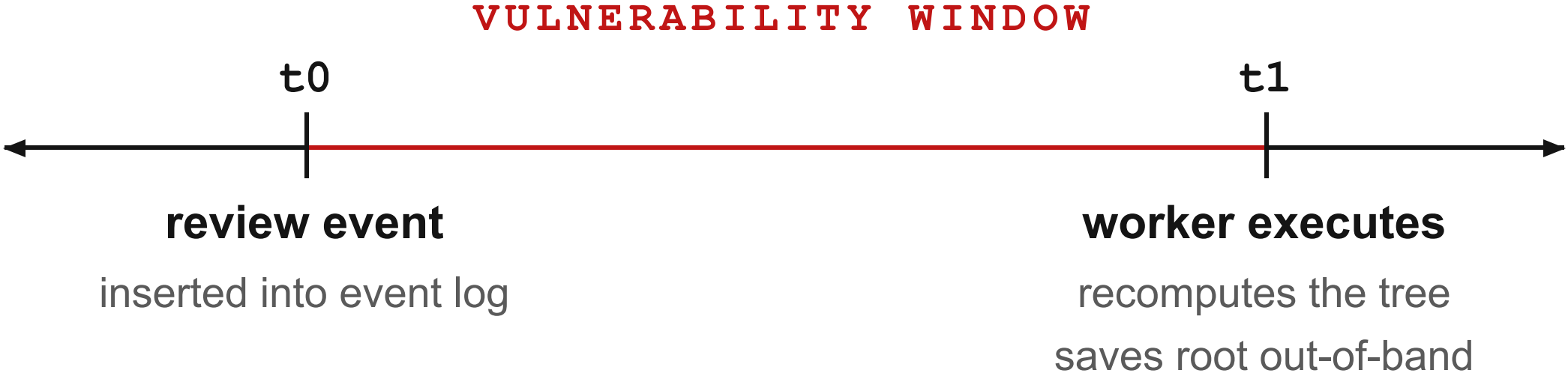


Signature algorithm → **ES256**

Signatures don't detect deletion; a Merkle log does



Events are unprotected until the worker commits them

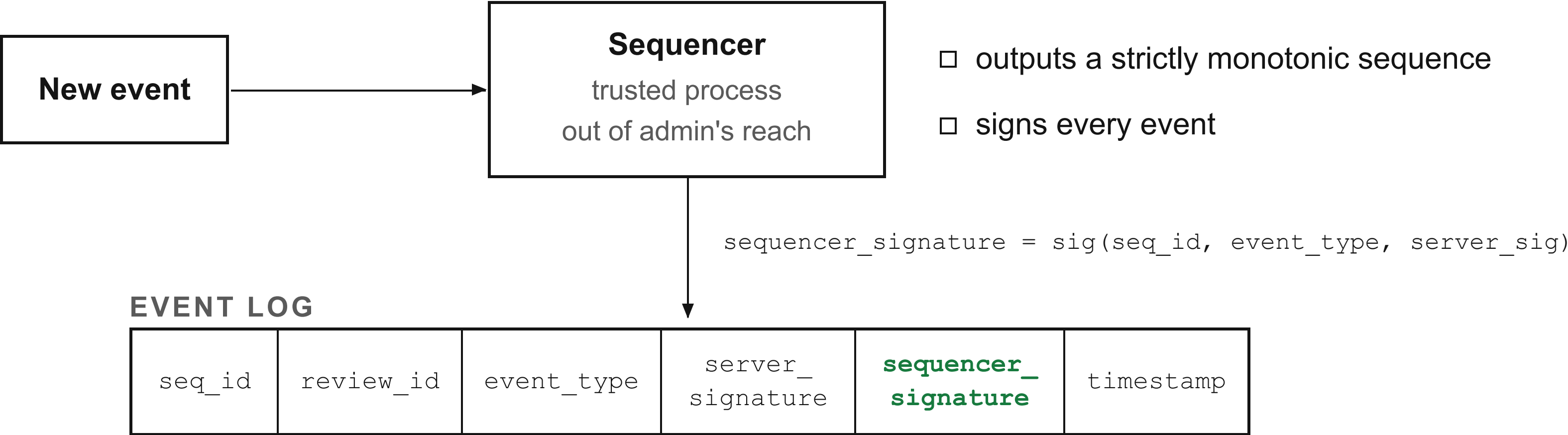


↑
admin can delete uncommitted entries in the event log

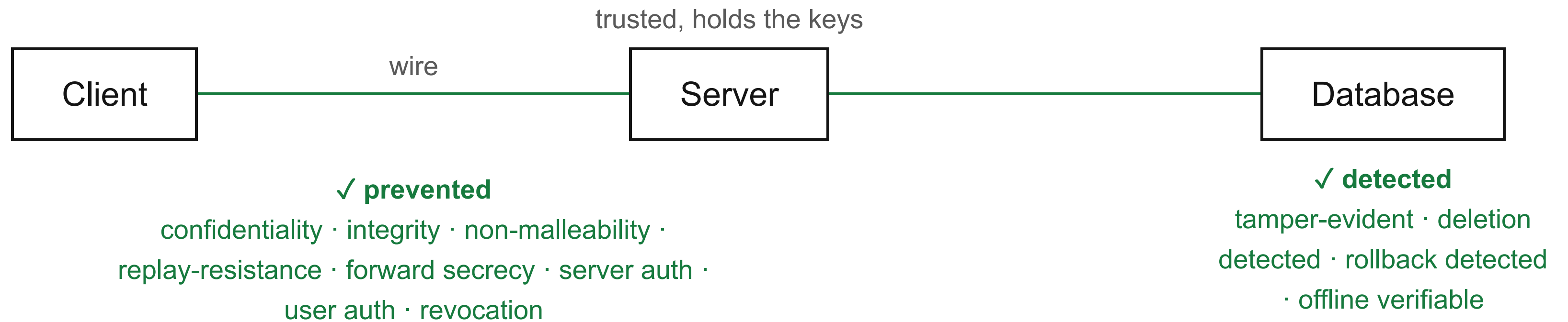
⋮
41 CREATE
42 MODIFY
43 CREATE
44 DELETE
⋮

← admin deletes an entry

An unreachable sequencer makes every missing event obvious



Prevented on the wire, detected at rest



Attack demo

- state rollback to an old signed version
- ledger tampering: relabel · delete · overwrite
- deletion in the uncommitted window
- signature forgery

Limitations and future work

- Extend coverage to movie & user data
- Protect the comments count
- Move to a distributed architecture
- Benchmark latency & dynamic batching
- Cache roots & richer Merkle trees